

LAB ASSIGNMENT

Build a Mini ETL Pipeline with Python

Extract → Transform → Load → Validate

Scenario

บริษัทจำลอง CampusMart มีข้อมูลจากหลายแหล่ง แต่ข้อมูลยังไม่พร้อมสำหรับวิเคราะห์

นิสิตต้องสร้าง ETL Pipeline ด้วย Python ให้ครบตั้งแต่ Extract → Transform → Load → Validate

ภารกิจหลัก

สร้าง Pipeline ที่อ่านข้อมูลหลายรูปแบบ
ทำความสะอาดและรวมข้อมูล แยก record ที่ผิดปกติ
โหลดเข้า SQLite Warehouse
และตรวจสอบผลลัพธ์ให้สามารถ rerun
ได้โดยไม่เกิดข้อมูลซ้ำ

Pipeline ที่ต้องสร้าง



PART 1 | Extract

คะแนน

Implement [src/extract.py](#)

ต้องอ่าน

- customers.csv
- orders.csv
- products.json
- ตาราง stores จาก store.db

สำหรับ JSON ให้ใช้ [pd.json_normalize\(\)](#) เพื่อ flatten nested JSON

ผลลัพธ์ที่คาดหวัง

```
raw = {  
    "customers": ...,  
    "orders": ...,  
    "products": ...,  
    "stores": ...  
}
```

Checkpoint หลัง Extract

ควรได้ DataFrame ครบ 4 ชุด และตรวจสอบ shape / columns เบื้องต้นก่อนเข้าสู่ Transform

Implement [src/transform.py](#)

Customers

- remove duplicate customer_id
- standardize province
- handle missing values

Products

- flatten JSON
- เปลี่ยนชื่อ column ให้ใช้ง่าย
- convert price เป็น numeric
- missing category → "Unknown"

Orders

- remove duplicate order_id
- parse mixed date formats
- standardize status เป็น lowercase
- reject record เมื่อ:
 - qty <= 0
 - unit_price <= 0
 - discount_pct < 0 หรือ > 100
 - date ไม่ถูกต้อง

Merge

เก็บเฉพาะ:

```
paid
completed
```

จากนั้น join:

```
orders
+ customers
+ products
```

ถ้า customer หรือ product ไม่มีใน master ให้ย้ายไป reject

คำนวณ

```
gross_amount = qty * unit_price
discount_amount = gross_amount * discount_pct / 100
sales_amount = gross_amount - discount_amount
```

สร้าง: [output/rejects.csv](#)

หลักคิดของ Transform

ข้อมูลผิดไม่ควรหายเงียบ ๆ: record
ที่ไม่ผ่านกฎต้องถูกแยกไว้ใน rejects
เพื่อให้ตรวจสอบย้อนหลังได้

PART 3 | Load

25

คะแนน

Implement [src/load.py](#)

Load เข้า SQLite:

data/warehouse/warehouse.db

ต้องมี 3 ตาราง

dim_customer

- customer_id
- name
- province
- email

dim_product

- product_id
- product_name
- category
- price

fact_sales

- order_id
- customer_id
- product_id
- order_date
- qty
- unit_price
- discount_pct
- sales_amount

Requirement สำคัญ

order_id ต้อง UNIQUE และเมื่อรัน Pipeline สองครั้ง จำนวน record ใน fact_sales ต้องเท่าเดิม

python -m src.main

PART 4 | Validate

15

คะแนน

Implement [src/validate.py](#)

ตรวจสอบอย่างน้อย

- จำนวน valid source rows
- จำนวน warehouse rows
- duplicate order_id
- total sales จาก transformed data
- total sales จาก warehouse
- PASS / FAIL

ตัวอย่าง validation.json

```
{
  "source_valid_rows": 92,
  "warehouse_rows": 92,
  "duplicate_order_ids": 0,
  "source_total_sales": 123456.78,
  "warehouse_total_sales": 123456.78,
  "status": "PASS"
}
```

Validation = หลักฐาน

Pipeline ที่รันจบไม่ได้แปลว่าถูกต้อง ต้องมีตัวเลขจาก transformed data และ warehouse ที่ตรวจสอบกันได้

การรัน

macOS / Linux

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

python -m src.main
```

Windows PowerShell

```
python -m venv .venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt

python -m src.main
```

สิ่งที่ต้องส่ง

ส่ง ZIP ชื่อ:

```
studentid_ETL_Lab.zip
```

- src/
- data/warehouse/warehouse.db
- output/rejects.csv
- output/validation.json
- logs/etl.log
- REPORT.md

ใน REPORT.md ตอบสั้น ๆ

1. พบ Data Quality Problem อะไรบ้าง
2. แก้แต่ละปัญหาอย่างไร
3. มี record ถูก reject กี่รายการ
4. ยอดขายรวมหลัง Transform เท่าไร
5. เมื่อลอง run pipeline เข้า fact_sales เพิ่มขึ้นหรือไม่ เพราะอะไร